

[school-of-solana](#) / [4.lesson](#) / 



 [hyckomatej](#) Update materials

03c5a68 · 3 months ago 

Name		Name	Last commit date
	..		
	bank	Update materials	3 months ago
	game	Update materials	3 months ago
	puppet	Update materials	3 months ago
	README.md	Update materials	3 months ago
school-of-solana / 4.lesson /			↑ Top
Name			

4. Lecture - Solana programming model II

Table of Contents

- [Cross-Program Invocations](#)
 - [Setting up basic CPI functionality](#)
 - [Privilege Extension](#)
 - [Reloading an Account](#)
 - [Returning values from handler functions](#)
 - [Note](#)
- [Program Derived Addresses](#)
 - [Creation of a PDA](#)
 - [Using PDAs](#)
 - [Hashmap-like structures using PDAs](#)
 - [Building hashmaps with PDAs](#)
 - [How to build PDA hashmaps in Anchor](#)
 - [Enforcing uniqueness](#)
 - [Conclusion](#)

Cross-Program Invocations

Often it's useful for programs to interact with each other. In Solana this is achieved via Cross-Program Invocations (CPIs).

Consider the following example of a puppet and a puppet master. Admittedly, it is not very realistic but it allows us to show you the many nuances of CPIs.

Setting up basic CPI functionality

Create a new workspace

```
anchor init puppet
```



and copy the following code.

```
use anchor_lang::prelude::*;

declare_id!("7kZGeneY2LYUhjPoveEkriYFWinHLbMfDbBQpcTvQbip");

// Defines the main functionalities of the puppet program.
#[program]
pub mod puppet {
    use super::*;

    // Function to initialize account data.
    pub fn initialize(_ctx: Context<Initialize>) -> Result<()> {
        Ok(())
    }

    // Function to set `data` on a puppet account.
    pub fn set_data(ctx: Context<SetData>, data: u64) -> Result<u64> {
        // Access the puppet account and update its `data`.
        let puppet = &mut ctx.accounts.puppet;
        puppet.data = data;
    }
}
```



```

        Ok(data)
    }
}

// Defines the account contexts for the `initialize` function.
#[derive(Accounts)]
pub struct Initialize<'info> {
    // Initializes a new `Data` account with space allocation for storage.
    #[account(
        init,
        payer = user,
        space = 8 + 8
    )] // 8 bytes for the discriminator, 8 for the data.
    pub puppet: Account<'info, Data>,
    // Marks the transaction's signer as the payer for the account creation.
    #[account(mut)]
    pub user: Signer<'info>,
    // Includes the System Program in the transaction, needed for account creation.
    pub system_program: Program<'info, System>,
}

// Defines the account contexts for the `set_data` function.
#[derive(Accounts)]
pub struct SetData<'info> {
    // References a mutable puppet account to update its data.
    #[account(mut)]
    pub puppet: Account<'info, Data>,
}

// Defines the data structure stored within a `puppet` account.
#[account]
pub struct Data {

```

```
    pub data: u64, // Storage for a 64-bit unsigned integer.
}
```

There's nothing special happening here. It's a pretty simple program! The interesting part is how it interacts with the next program we are going to create.

Still, inside the project, initialize a new `puppet_master` program using,

```
anchor new puppet_master
```



inside the workspace and copy the following code.

```
use anchor_lang::prelude::*;
use puppet::cpi::accounts::SetData;
use puppet::program::Puppet;
use puppet::{self, Data};

declare_id!("BGzJov5eNHhCU6vUgitmuLxv91nxRMjzt9zpWMDhF22T");

// Defines the main functionalities of the puppet_master program.
#[program]
mod puppet_master {
    use super::*;

    // Function to manipulate data on a puppet account via a cross-program invocation (CPI).
    pub fn pull_strings(ctx: Context<PullStrings>, data: u64) -> Result<()> {
        // Convert the puppet_program account into account info for the CPI call.
        let cpi_program = ctx.accounts.puppet_program.to_account_info();
        // Set up the CPI accounts expected by the puppet::set_data function.
```



```

    let cpi_accounts = SetData {
        puppet: ctx.accounts.puppet.to_account_info(),
    };
    // Create a CPI context with the program and accounts to call.
    let cpi_ctx = CpiContext::new(cpi_program, cpi_accounts);
    // Perform the CPI call to set data on the puppet account.
    let result = puppet::cpi::set_data(cpi_ctx, data)?;
    Ok(())
}
}

// Specifies the accounts required for the `pull_strings` function.
#[derive(Accounts)]
pub struct PullStrings<'info> {
    // References a mutable puppet account to manipulate its data.
    #[account(mut)]
    pub puppet: Account<'info, Data>,
    // Reference to the puppet program to enable CPI calls to it.
    pub puppet_program: Program<'info, Puppet>,
}

```

Make sure that all of the Program IDs match. This means, when you run `anchor keys list`, the output of the command has to match with the Program IDs specified inside `Anchor.toml` and also with the Program IDs used within the `declare_id!` macro of both programs. If by any chance these do not match, use Program IDs from the `anchor keys list` as a reference (i.e. change `declare_id!` accordingly). Finally, import the `puppet` program into the `puppet_master` program by adding the following line to the `[dependencies]` section of the `Cargo.toml` file inside the `puppet_master` program folder:

```
puppet = { path = "../puppet", features = ["cpi"] }
```



The `features = ["cpi"]` is used so we can not only use puppet's types but also its instruction builders and CPI functions. Without those, we would have to use low level solana syscalls. Fortunately, Anchor provides abstractions on top of those. By enabling the cpi feature, the puppet_master program gets access to the `puppet::cpi` module. Anchor generates this module automatically and it contains tailor-made instructions builders and CPI helpers for the program.

In the case of the puppet program, the puppet_master uses the `SetData` instruction builder struct provided by the `puppet::cpi::accounts` module to submit the accounts the `SetData` instruction of the puppet program expects. Then, the puppet_master creates a new CPI context and passes it to the `puppet::cpi::set_data` cpi function. This function has the exact same function arguments as the `set_data` function in the puppet program with the exception that it expects a `CpiContext` instead of a `Context`.

We can verify that everything works as expected by replacing the contents of the `puppet.ts` file with the following code and running `anchor test`.

```
// Import required modules and classes from Anchor and Solana's web3.js
import * as anchor from '@coral-xyz/anchor';
import { Program } from '@coral-xyz/anchor';
import { Keypair } from '@solana/web3.js';
import { assert } from 'chai';
import { Puppet } from '../target/types/puppet';
import { PuppetMaster } from '../target/types/puppet_master';

// Define a test suite for the Puppet program.
describe('puppet', () => {
  // Initialize the provider to interact with the Solana network.
  const provider = anchor.AnchorProvider.env();
  anchor.setProvider(provider);

  // Reference the deployed Puppet and PuppetMaster programs for testing.
  const puppetProgram = anchor.workspace.Puppet as Program<Puppet>;
```



```
const puppetMasterProgram = anchor.workspace.PuppetMaster as Program<PuppetMaster>;

// Generate a new keypair for the puppet account to be used in tests.
const puppetKeypair = Keypair.generate();

// Define a test for Cross-Program Invocation (CPI) between Puppet and PuppetMaster.
it('Does CPI!', async () => {
  // Initialize the puppet account with the generated keypair.
  await puppetProgram.methods
    .initialize()
    .accounts({
      puppet: puppetKeypair.publicKey,
      user: provider.wallet.publicKey,
    })
    .signers([puppetKeypair])
    .rpc();

  // Call the pullStrings method of the PuppetMaster program to manipulate puppet data.
  await puppetMasterProgram.methods
    .pullStrings(new anchor.BN(42)) // BN represents a BigNumber for handling large integers.
    .accounts({
      puppetProgram: puppetProgram.programId,
      puppet: puppetKeypair.publicKey,
    })
    .rpc();

  // Fetch the updated data from the puppet account to verify the change.
  const puppetData = await puppetProgram.account.data.fetch(puppetKeypair.publicKey);

  // Assert that the data was correctly set to 42 by the CPI call.
  assert.equal(puppetData.data.toNumber(), 42);
```



```
});  
});
```

Privilege Extension

CPIs extend the privileges of the caller to the callee. The puppet account was passed as a mutable account to the puppet_master but it was still mutable in the puppet program as well (otherwise the `assert` in the test would've failed). The same applies to signatures.

⚠ Warning

Privilege extension is convenient but also dangerous. If a CPI is unintentionally made to a malicious program, this program has the same privileges as the caller. Anchor protects you from CPIs to malicious programs with two measures. First, the `Program<'info, T>` type checks that the given account is the expected program `T`. Should you ever forget to use the `Program` type, the automatically generated cpi function (in the previous example this was `puppet::cpi::set_data`) also checks that the `cpi_program` argument equals the expected program.

Reloading an Account

In the puppet program, the `Account<'info, T>` type is used for the puppet account. If a CPI edits an account of that type, the caller's account does not change during the instruction.

You can easily see this for yourself by adding the following right after the `puppet::cpi::set_data(ctx.accounts.set_data_ctx(), data)` cpi call.

```
puppet::cpi::set_data(ctx.accounts.set_data_ctx(), data)?;  
if ctx.accounts.puppet.data != 42 {  
  panic();  
}
```



```
}  
Ok(())
```

The reason the data field has not been updated to 42 in the caller is that at the beginning of the instruction the `Account<'info, T>` type deserializes the incoming bytes into a new struct. This struct is no longer connected to the underlying data in the account. The CPI changes the data in the underlying account but since the struct in the caller has no connection to the underlying account the struct in the caller remains unchanged.

If you need to read the value of an account that has just been changed by a CPI, you can call its reload method which will re-deserialize the account. If you put `ctx.accounts.puppet.reload()?;` right after the CPI call, the test will pass.

```
puppet::cpi::set_data(ctx.accounts.set_data_ctx(), data)?;  
ctx.accounts.puppet.reload()?;  
if ctx.accounts.puppet.data != 42 {  
    panic!();  
}  
Ok(())
```



Returning values from handler functions

The Anchor handler functions are capable of returning data using the Solana `set_return_data` and `get_return_data` syscalls. This data can be used in CPI callers and clients.

Instead of returning a `Result<>`, consider this version of the `set_data` function from above which has been modified to return `Result<u64>`:

```
pub fn set_data(ctx: Context<SetData>, data: u64) -> Result<u64> {  
    let puppet = &mut ctx.accounts.puppet;
```



```

    puppet.data = data;
    Ok(data)
}

```

Defining a return type that isn't the unit type `()` will cause Anchor to transparently call `set_return_data` with the given type (`u64` in this example) when this function is called. The return from the CPI call is wrapped in a struct to allow for lazy retrieval of this return data. E.g.

```

pub fn pull_strings(ctx: Context<PullStrings>, data: u64) -> Result<()> {
    let cpi_program = ctx.accounts.puppet_program.to_account_info();
    let cpi_accounts = SetData {
        puppet: ctx.accounts.puppet.to_account_info(),
    };
    let cpi_ctx = CpiContext::new(cpi_program, cpi_accounts);
    let result = puppet::cpi::set_data(cpi_ctx, data)?;
    // The below statement calls sol_get_return and deserializes the result.
    // `return_data` contains the return from `set_data`,
    // which in this example is just `data`.
    let return_data = result.get();
    // ... do something with the `return_data` ...
}

```

Note

The type being returned must implement the `AnchorSerialize` and `AnchorDeserialize` traits, so for custom structure types and enums you need to do for example:

```

#[derive(AnchorSerialize, AnchorDeserialize)]
pub struct StructReturn {

```

```
pub value: u64,  
}
```

Program Derived Addresses

Knowing how to use PDAs is one of the most important skills for Solana Programming. They simplify the programming model and make programs more secure. So what are they?

PDAs (program derived addresses) are addresses with special properties.

Unlike normal addresses, PDAs are not public keys and therefore do not have an associated private key. There are two use cases for PDAs. They provide a mechanism to build hashmap-like structures on-chain and they allow programs to sign instructions.

Creation of a PDA

Before we dive into how to use PDAs in anchor, here's a short explainer on what PDAs are.

PDAs are created by hashing a number of **seeds** the user can choose and the **id** of a program:

```
// pseudo code  
let pda = hash(seeds, program_id);
```



There's a 50% chance that this hash function results in a public key (but PDAs are not public keys), so a **bump** has to be searched for so that we get a PDA:



```
// pseudo code
fn find_pda(seeds, program_id) {
  for bump in 0..256 {
    let potential_pda = hash(seeds, bump, program_id);
    if is_pubkey(potential_pda) {
      continue;
    }
    return (potential_pda, bump);
  }
  panic!("Could not find pda after 256 tries.");
}
```

It is technically possible that no bump is found within 256 tries but this probability is negligible. If you're interested in the exact calculation of a PDA, check out the [solana_program source code](#).

The first bump that results in a PDA is commonly called the "**canonical bump**". Other bumps may also result in a PDA but it's recommended to only use the canonical bump to avoid confusion.

Using PDAs

We are now going to show you what you can do with PDAs and how to do it in Anchor!

HashMap-like structures using PDAs

Before we dive into the specifics of creating hashmaps in anchor, let's look at how to create a hashmap with PDAs in general.

Building hashmaps with PDAs

PDA's are hashed from the **bump**, a **program id**, but also a number of **seeds** which can be freely chosen by the user. These seeds can be used to build hashmap-like structures on-chain.

For instance, imagine you're building an in-browser game and want to store some user stats. Maybe their level and their in-game name. You could create an account with a layout that looks like this:

```
pub struct UserStats {  
  level: u16,  
  name: String,  
  authority: Pubkey  
}
```



The authority would be the user the accounts belongs to.

This approach creates the following problem. It's easy to go from the user stats account to the user account address (just read the authority field) but if you just have the user account address (which is more likely), how do you find the user stats account? You can't. This is a problem because your game probably has instructions that require both the user stats account and its authority which means the client needs to pass those accounts into the instruction (for example, a `ChangeName` instruction). So maybe the frontend could store a mapping between a user's account address and a user's info address in local storage. This works until the user accidentally wipes their local storage.

With PDA's you can have a layout like this:

```
pub struct UserStats {  
  level: u16,  
  name: String,  
  bump: u8  
}
```



The information about the relationship between the user and the user stats account is encoded in the address of the user stats account itself.

Reusing the pseudo code from above:

```
// pseudo code
let seeds = [b"user-stats", authority];
let (pda, bump) = find_pda(seeds, game_program_id);
```



When a user connects to your website, this pda calculation can be done client-side using their user account address as the authority. The resulting PDA then serves as the address of the user's stats account. `b"user-stats"` is added in case there are other account types that are also PDAs. If there were an inventory account, it could be inferred using these seeds:

```
let seeds = [b"inventory", authority];
```



To summarize, we have used PDAs to create a mapping between a user and their user stats account. There is no single hashmap object that exposes a get function. Instead, each value (the user stats address) can be found by using certain seeds ("user-stats" and the user account address) as inputs to the `find_pda` function.

How to build PDA hashmaps in Anchor

Let's create a new workspace:

```
anchor init game
```



and copy the following code



```
use anchor_lang::prelude::*;

declare_id!("Fg6PaFpoGXkYsidMpWTK6W2BeZ7FEfcYkg476zPFsLnS");

// Defines the `game` module containing the business logic.
#[program]
pub mod game {
    use super::*;

    // Function to create user statistics and initialize an account.
    pub fn create_user_stats(ctx: Context<CreateUserStats>, name: String) -> Result<()> {
        // Access and modify the user_stats account.
        let user_stats = &mut ctx.accounts.user_stats;
        user_stats.level = 0; // Set initial level to 0.
        if name.as_bytes().len() > 200 {
            // If the name exceeds 200 bytes, terminate with an error.
            panic!(); // Error handling simplified for brevity.
        }
        user_stats.name = name; // Store the name in the user_stats account.
        user_stats.bump = ctx.bumps.user_stats; // Safely store the bump used for this account.
        Ok(())
    }
}

// Data structure to store user statistics.
#[account]
pub struct UserStats {
    level: u16, // User's level, stored as a 16-bit unsigned integer.
    name: String, // User's name, stored as a dynamically sized string.
    bump: u8, // Bump seed for address derivation.
}
```



```

}

// Accounts context structure for creating user statistics.
#[derive(Accounts)]
pub struct CreateUserStats<'info> {
    #[account(mut)]
    pub user: Signer<'info>, // Signer who pays for the account creation and processing.
    // Initializes a user_stats account with predefined space allocation.
    #[account(
        init,
        payer = user,
        space = 8 + 2 + 4 + 200 + 1, // Account size calculation includes all fields.
        seeds = [b"user-stats", user.key().as_ref()], // Seeds for deterministic address.
        bump
    )]
    pub user_stats: Account<'info, UserStats>, // The user_stats account to be created.
    pub system_program: Program<'info, System>, // System program to handle account creation.
}

```

In the account validation struct we use `seeds` together with `init` to create a PDA with the desired seeds. Additionally, we add an empty `bump` constraint to signal to anchor that it should find the canonical bump itself. Then, in the handler, we call `ctx.bumps.user_stats` to get the bump anchor found and save it to the user stats account as an extra property.

If we then want to use the created PDA in a different instruction, we can add a new validation struct (This will check that the `user_stats` account is the pda created by running `hash(seeds, user_stats.bump, game_program_id)`):

```

// Defines the accounts context for the `change_user_name` function in the smart contract.
#[derive(Accounts)]
pub struct ChangeUserName<'info> {
    // Signer of the transaction, who is assumed to be the user making the change.

```



```

pub user: Signer<'info>,
// The `user_stats` account is required to be mutable because its data will be updated.
// This account is looked up using seeds comprising a static "user-stats" seed and the user's public key
// ensuring that the account is unique per user.
#[account(mut, seeds = [b"user-stats", user.key().as_ref()], bump = user_stats.bump)]
pub user_stats: Account<'info, UserStats>,
}

```

and another handler function:

```

// handler function (add this next to the create_user_stats function in the game module)
pub fn change_user_name(ctx: Context<ChangeUserName>, new_name: String) -> Result<()> {
    if new_name.as_bytes().len() > 200 {
        // proper error handling omitted for brevity
        panic!();
    }
    ctx.accounts.user_stats.name = new_name;
    Ok(())
}

```

Finally, consider the following game.ts test file:

```

// Import required modules from Anchor and Solana's web3.js.
import * as anchor from '@coral-xyz/anchor';
import { Program } from '@coral-xyz/anchor';
import { PublicKey } from '@solana/web3.js';
import { Game } from '../target/types/game';
import { expect } from 'chai';

```

```
// Define a test suite for the Game program.
describe('game', async () => {
  // Initialize the provider to interact with the Solana network.
  const provider = anchor.AnchorProvider.env();
  anchor.setProvider(provider);

  // Reference the deployed Game program for testing.
  const program = anchor.workspace.Game as Program<Game>;

  // Test to verify name setting and changing functionality.
  it('Sets and changes name!', async () => {
    // Find the Program Derived Address (PDA) for the userStats account.
    const [userStatsPDA, _] = await PublicKey.findProgramAddressSync(
      [
        anchor.utils.bytes.utf8.encode('user-stats'), // Encode the seed for the PDA.
        provider.wallet.publicKey.toBuffer(), // Include the public key of the wallet as part of the seed.
      ],
      program.programId // Specify the program ID used for generating the PDA.
    );

    // Create user stats with the name 'brian'.
    await program.methods
      .createUserStats('brian')
      .accounts({
        user: provider.wallet.publicKey, // Specify the user account as the payer.
        userStats: userStatsPDA, // Specify the PDA as the userStats account.
      })
      .rpc(); // Perform the RPC call to execute the method on-chain.

    // Fetch the userStats account and assert that the name is correctly set to 'brian'.
    expect((await program.account.userStats.fetch(userStatsPDA)).name).toEqual('brian');

    // Change the name in the userStats account to 'tom'.
```

```
await program.methods
  .changeUserName('tom')
  .accounts({
    user: provider.wallet.publicKey, // Specify the user account as the payer.
    userStats: userStatsPDA, // Specify the PDA as the userStats account.
  })
  .rpc(); // Perform the RPC call to execute the method on-chain.

// Fetch the userStats account again and assert that the name is changed to 'tom'.
expect((await program.account.userStats.fetch(userStatsPDA)).name).to.equal('tom');
});
});
```

Exactly as described in the subchapter before this one, we use a find function to find the PDA. We can then use it just like a normal address. Well, almost. When we call `createUserStats`, we don't have to add the PDA to the `[signers]` array even though account creation requires a signature. This is because it is impossible to sign the transaction from outside the program as the PDA (it's not a public key so there is no private key to sign with). Instead, the signature is added when the CPI to the system program is made.

Enforcing uniqueness

A subtle result of this hashmap structure is enforced uniqueness. When `init` is used with **seeds** and **bump**, it will always search for the **canonical bump**. This means that it can only be called once (because the 2nd time it's called the PDA will already be initialized). To illustrate how powerful enforced uniqueness is, consider a decentralized exchange program. In this program, anyone can create a new market for two assets. However, the program creators want liquidity to be concentrated so there should only be one market for every combination of two assets. This could be done without PDAs but would require a global account that saves all the different markets. Then upon market creation, the program would check whether the asset combination exists in the global market list. With PDAs this can be done in a much more straightforward way. Any market would simply be the PDA of the mint addresses of the two assets. The program would then check whether either of the two possible PDAs (because the market could've been created with the assets in reverse order) already exists.

Conclusion

This section serves as a brief recap of the different things you can do with PDAs.

1. **Create hashmaps:** We created a user stats PDA which was derived from the user address. This derivation linked the user address and the user stats account, allowing the latter to be easily found given the former. Hashmaps also result in enforced uniqueness which can be used in many different ways, e.g. for only allowing one market per two assets in a decentralized exchange.
2. **Allow programs to sign CPIs:** This means that programs can be given control over assets which they then manage according to the rules defined in their code.

You can even combine these two use cases and use a PDA that's used in an instruction as a state account to also sign a CPI.

Need help?

If you have any questions feel free to reach out to us on [Discord](#).